DIP Session#2

CSE 4246:
Digital Image Processing Sessional

Dr. S.M. Jahangir Alam (JA)
Professor, Dept. of CSE

Contents

DIP Sessional#02:

Module#2.1: Image Enhancement: Image Contrast, Image Histogram, CLAHE, Histogram Equalization, Matching, Brightness, Contrast, Gamma, Negative

Module#2.2: Image Transformation: Crop, Resize, Rotate, Flip

Image Contrast

In image processing, **contrast** is the difference in intensity (brightness) or color between pixels in an image such as color, brightness, size, shape, or meaning.

$$g(x,y)=\alpha f(x,y)+\beta$$

- α = Controls Contrast
- β = Controls Brightness

α < 1	Decreases contrast	β < 0	Darkens image
α = 1	No contrast change	β = 0	No brightness change
α > 1	Increases contrast	β > 0	Brightens image







Automatic contrast stretching

$$\alpha = \frac{255}{r_{max} - r_{min}}$$

$$\beta = -\alpha r_{min}$$

$$g = \alpha f+\beta$$

Image Contrast

1. Contrast using intensity range

$$Contrast = f_{max} - f_{min}$$

2. Michelson contrast:

$$Contrast = \frac{f_{max} - f_{min}}{f_{max} + f_{min}}$$

3. RMS contrast

$$C_{RMS} = \sqrt{\frac{1}{MN}\sum_{x,y}(f(x,y) - \mu)^2}$$







Image Contrast

```
Img_path=r"D:/DIPLab/Image/Multicolor-MagnificFlower.bmp";

img = cv2.imread(Img_path)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

 contrast_low = cv2.convertScaleAbs(img, alpha=0.5, beta=0)

 contrast_high = cv2.convertScaleAbs(img, alpha=1.8, beta=0)

 gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

 plt.imshow(gray, cmap='gray')
```

Image Contrast

- import cv2
- import numpy as np
- import matplotlib.pyplot as plt

- # Read image in grayscale
- Img_path=r"D:/DIPLab/Image/Multicolor-MagnificFlower.bmp";

- img = cv2.imread(Img_path)

- if img is None:
- print("Image not found!")
- else:
- img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

Image Contrast

- # Calculate Contrast
- I_max = np.max(img)
- I_min = np.min(img)
- mean = np.mean(img)

- # Simple Contrast
- simple_contrast = I_max - I_min

- # Michelson Contrast
- michelson = (I_max - I_min) / (I_max + I_min)

- # RMS Contrast
- rms = np.sqrt(np.mean((img - mean) ** 2))

Image Contrast

- # Display Image and Histogram
- plt.figure(figsize=(10,4))

- # Original Image
- plt.subplot(1,2,1)
- plt.imshow(img, cmap='gray')
- plt.title("Original Image")
- plt.axis("off")

```
# Histogram
plt.subplot(1,2,2)
plt.hist(img.ravel(), bins=20,
range=(0,20), color='red')
plt.title("Image Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```



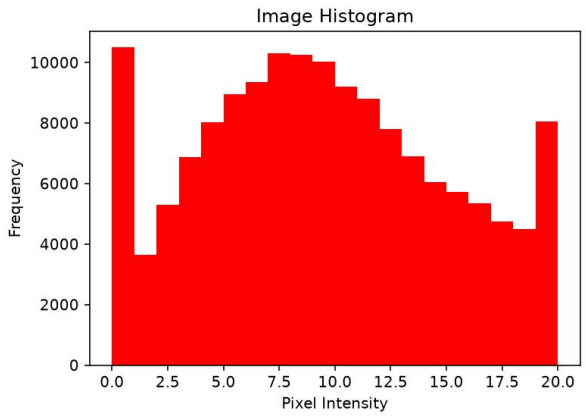


Image Contrast

- # Print Results
- print("Maximum Intensity :", I_max)
- print("Minimum Intensity :", I_min)
- print("Mean Intensity   :", mean)

- print("\nSimple Contrast   :", simple_contrast)
- print("Michelson Contrast:", round(michelson,4))
- print("RMS Contrast   :", round(rms,4))

Image Contrast

```
hist = cv2.calcHist([gray], [0], None, [256], [0, 220])

equalized = cv2.equalizeHist(gray)

clahe = cv2.createCLAHE(clipLimit=20.0, tileGridSize=(8,8))

result = clahe.apply(gray)

clahe_hist = cv2.calcHist([result], [0], None, [256], [0, 220])
```

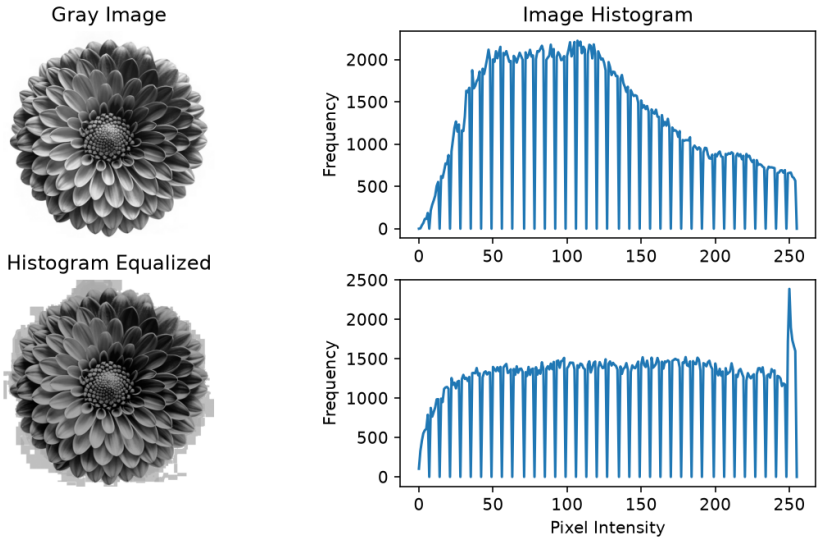
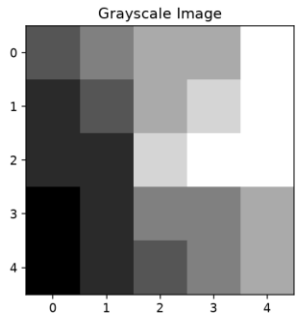
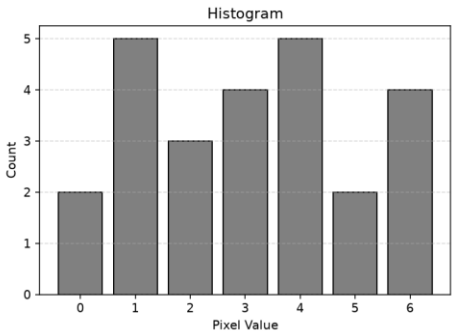


Image Histogram

- import numpy as np
- import matplotlib.pyplot as plt





- # 5×5 grayscale image
- image = np.array([
- [2, 3, 4, 4, 6],
- [1, 2, 4, 5, 6],
- [1, 1, 5, 6, 6],
- [0, 1, 3, 3, 4],
- [0, 1, 2, 3, 4]
-])

```
print("Histogram")
print("---------------")
print("Pixel Value\tCount")
for i, count in enumerate(hist):
    print(f"{i}\t\t{count}")

# Display Image and Histogram
plt.figure(figsize=(10,4))
```

Calculate histogram

```
hist = np.bincount(image.flatten(), minlength=7)
```

Image Histogram

Image

```
plt.subplot(1,2,1)
plt.imshow(image, cmap='gray', vmin=0, vmax=6)
plt.title("Grayscale Image")
plt.xticks(range(5))
plt.yticks(range(5))
```

Histogram

- plt.subplot(1,2,2)
- plt.bar(range(len(hist)), hist, width=0.8, color='gray', edgecolor='black')
- plt.title("Histogram")
- plt.xlabel("Pixel Value")
- plt.ylabel("Count")
- plt.xticks(range(7))
- plt.grid(axis='y', linestyle='--', alpha=0.5)
- plt.tight_layout()
- plt.show()

Image contrast/histogram stretching

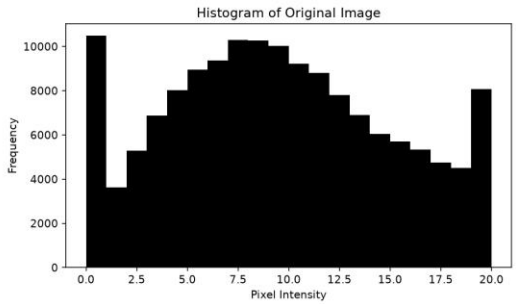
$$I_{new}(r,c) = \left[\frac{I(\mathrm{r},\mathrm{c}) - I_{min}(r,c)}{I_{max}(r,c) - I_{min}(r,c)}\right] \times 255$$

```
# Read grayscale image
Img_path=r"D:/DIPLab/Image/Multicolor-MagnificFlower.bmp";

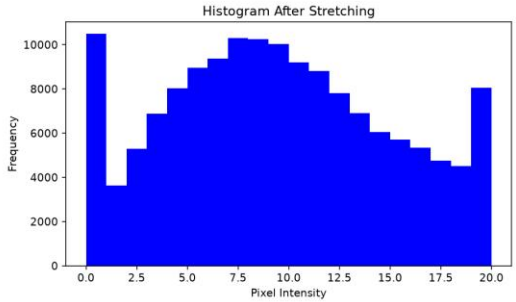
image = cv2.imread(Img_path)

if image is None:
    print("Image not found!")
else:
    img = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```










```
# Find minimum and maximum intensity
r_min = np.min(image)
r_max = np.max(image)
print("Minimum Intensity =", r_min)
print("Maximum Intensity =", r_max)
```

Image contrast/histogram stretching

$$I_{new}(r,c) = \left[\frac{I(\mathrm{r},\mathrm{c}) - I_{min}(r,c)}{I_{max}(r,c) - I_{min}(r,c)}\right] \times 255$$

Contrast Stretching

stretched = ((image - r_min) / (r_max - r_min) * 255).astype(np.uint8)

Plot Results

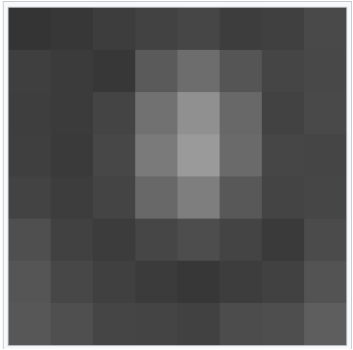
plt.figure(figsize=(12,8))

Original Image

plt.subplot(2,2,1)
plt.imshow(image, cmap='gray')
plt.title("Original Low Contrast Image")
plt.axis("off")

Normalized Histogram Equalization

##The 8x8 sub-image shown in 8-bit grayscale. The 8-bit grayscale image shown has the values.



52	55	61	59	79	61	76	61
62	59	55	104	94	85	59	71
63	65	66	113	144	104	63	72
64	70	70	126	154	109	71	69
67	73	68	106	122	88	68	68
68	79	60	70	77	66	58	75
69	85	64	58	55	61	65	83
70	87	69	68	65	73	78	90

Value	Count	Value	Count	Value	Count	Value	Count	Value	Count
52	1	64	2	72	1	85	2	113	1
55	3	65	3	73	2	87	1	122	1
58	2	66	2	75	1	88	1	126	1
59	3	67	1	76	1	90	1	144	1
60	1	68	5	77	1	94	1	154	1
61	4	69	3	78	1	104	2		
62	1	70	4	79	2	106	1		
63	2	71	2	83	1	109	1		

$$h(v) = round\left(\frac{cdf(v) - cdf_{min}}{(M \times N) - cdf_{min}} \times (L-1)\right)$$

$h(v) = round\left(\frac{1-1}{64-1} \times (256-1)\right)$=0, $round\left(\frac{4-1}{64-1} \times (255)\right)$=12,

$round\left(\frac{6-1}{64-1} \times (255)\right)$=20,

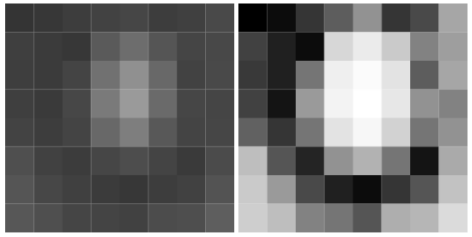
$round\left(\frac{9-1}{64-1} \times (255)\right)$=32, ⋯. and so on.

Normalized Histogram Equalization

V, Pixel Intensity	Cdf(v)	h(v), Equalized v
52	1	0
55	4	12
58	6	20
59	9	32
60	10	36
61	14	53
62	15	57
63	17	65
64	19	73
65	22	85
66	24	93
67	25	97
68	30	117

V, Pixel Intensity	Cdf(v)	h(v), Equalized v
69	33	130
70	37	146
71	39	154
72	40	158
73	42	166
75	43	170
76	44	174
77	45	178
78	46	182
79	48	190
83	49	194
85	51	202
87	52	206

V, Pixel Intensity	Cdf(v)	h(v), Equalized v
88	53	210
90	54	215
94	55	219
104	57	227
106	58	231
109	59	235
113	60	239
122	61	243
126	62	247
144	63	251
154	64	255



0	12	53	32	190	53	174	53
57	32	12	227	219	202	32	154
65	85	93	239	251	227	65	158
73	146	146	247	255	235	154	130
97	166	117	231	243	210	117	117
117	190	36	146	178	93	20	170
130	202	73	20	12	53	85	194
146	206	130	117	85	166	182	215

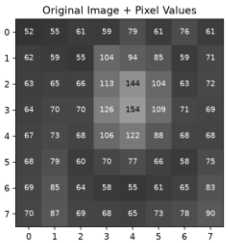
Normalized Histogram Equalization

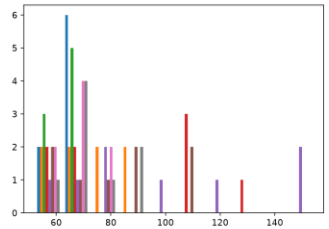
```
# 8×8 pixel intensity matrix
pixels = np.array([
[52, 55, 61, 59, 79, 61, 76, 61],
[62, 59, 55, 104, 94, 85, 59, 71],
[63, 65, 66, 113, 144, 104, 63, 72],
[64, 70, 70, 126, 154, 109, 71, 69],
[67, 73, 68, 106, 122, 88, 68, 68],
[68, 79, 60, 70, 77, 66, 58, 75],
[69, 85, 64, 58, 55, 61, 65, 83],
[70, 87, 69, 68, 65, 73, 78, 90] ],
dtype=np.uint8)
```

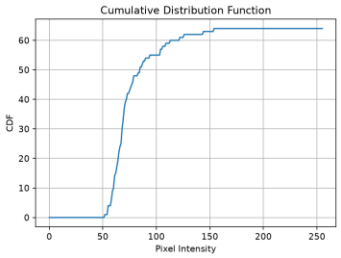
```
#Show pixels
print("Pixel Values:")
print(pixels)

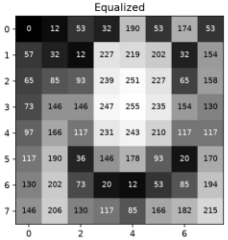
# Number of gray levels
L = 256

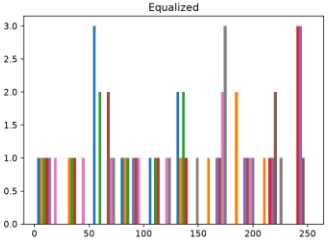
# Total number of pixels
MN = pixels.size
```












```
# Display as grayscale image
plt.figure(figsize=(8, 8))
plt.imshow(pixels,
cmap='gray', vmin=0,
vmax=255)
```

Normalized Histogram Equalization

```
# Histogram
hist = np.bincount(pixels.ravel(),minlength=L)

# Cumulative frequency
cdf = np.cumsum(hist)

# Minimum non-zero CDF
cdf_min = cdf[cdf > 0][0]

print("Total Pixels =", MN)
print("CDF_min =", cdf_min)

# Normalized histogram equalization
mapping = np.round(((cdf - cdf_min) /(MN - cdf_min)) * (L - 1))
```

Normalized Histogram Equalization

```
# Keep values between 0 and 255
mapping = np.clip(mapping, 0, 255).astype(np.uint8)

# Apply mapping
equalized = mapping[pixels]

print("\nOriginal Image:")
print(pixels)

print("\nNormalized Equalized Image:")
print(equalized)

print("Histogram:")
print(hist)
```

Normalized Histogram Equalization

```
# 8. Four Plots

plt.figure(figsize=(16, 8))

plt.subplot(2,3,1)
plt.imshow(pixels, cmap='gray', vmin=0, vmax=255)

for i in range(pixels.shape[0]):
    for j in range(pixels.shape[1]):

        value = int(pixels[i, j])

        if value > 127:
            text_color = "black"
        else:
            text_color = "white"
        plt.text(j, i, str(value), ha="center",  va="center",
                         color=text_color, fontsize=9 )
```

```
plt.xticks(range(8))
plt.yticks(range(8))
plt.title("Original Image + Pixel Values")
#plt.grid()

plt.subplot(2,3,2)
plt.hist(pixels)
```

Normalized Histogram Equalization

```
# Print CDF for values that actually occur
for intensity in np.unique(pixels):
    print(
        f"Intensity = {intensity:3d}, "
        f"Frequency = {hist[intensity]:2d}, "
        f"CDF = {cdf[intensity]:.4f}“
            )

# Plot CDF
plt.subplot(2,3,3)
plt.plot(cdf)
plt.xlabel("Pixel Intensity")
plt.ylabel("CDF")
plt.title("Cumulative Distribution Function")
plt.grid()
```

Normalized Histogram Equalization

```
plt.tight_layout()

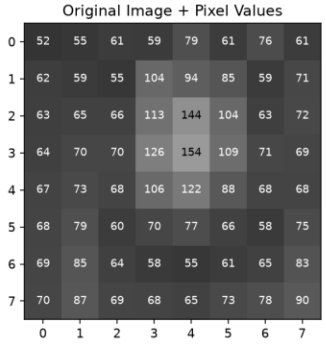
plt.subplot(2,3,4)
plt.imshow(equalized, cmap='gray', vmin=0, vmax=255)
plt.title("Equalized")
for i in range(equalized.shape[0]):
    for j in range(equalized.shape[1]):
        value = int(equalized[i, j])

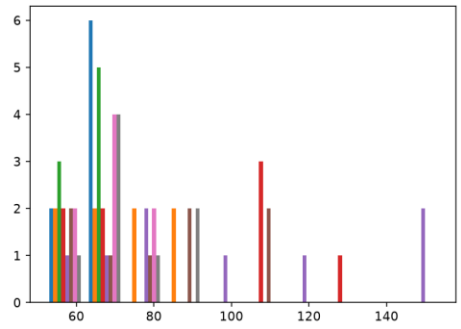
        if value > 127:
            text_color = "black"
        else:
            text_color = "white"
        plt.text( j, i,  str(value),  ha="center",   va="center",
            color=text_color,   fontsize=9)
```

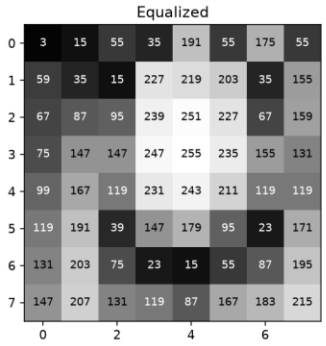
```
plt.subplot(2,3,5)
plt.hist(equalized)
plt.title("Equalized")

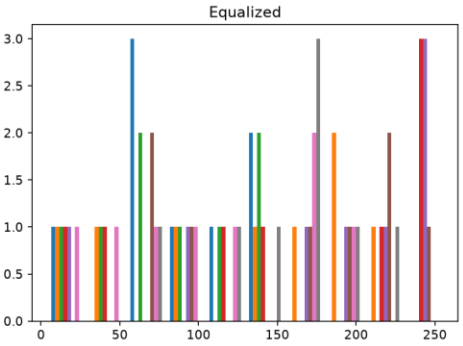
plt.tight_layout()
plt.show()
```

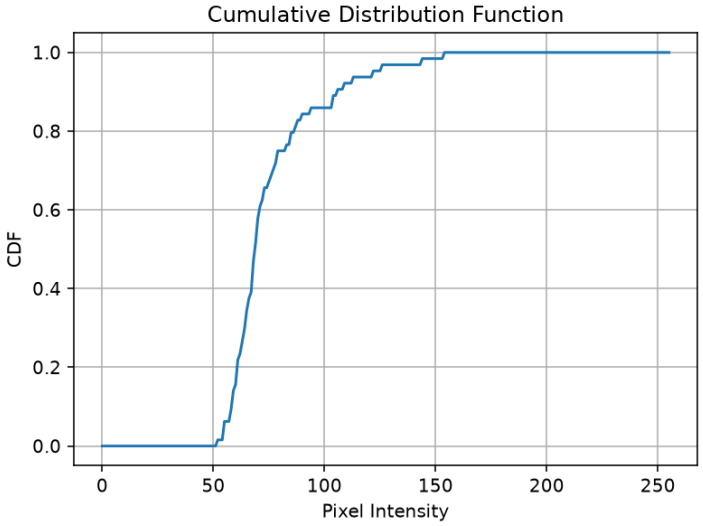
Basic Histogram Equalization












```
import numpy as np
import matplotlib.pyplot as plt

# 8×8 pixel intensity matrix
pixels = np.array([
[52, 55, 61, 59, 79, 61, 76, 61],
[62, 59, 55, 104, 94, 85, 59, 71],
[63, 65, 66, 113, 144, 104, 63, 72],
[64, 70, 70, 126, 154, 109, 71, 69],
[67, 73, 68, 106, 122, 88, 68, 68],
[68, 79, 60, 70, 77, 66, 58, 75],
[69, 85, 64, 58, 55, 61, 65, 83],
[70, 87, 69, 68, 65, 73, 78, 90] ],
dtype=np.uint8)
```

Basic Histogram Equalization

```
#Show pixels
print("Pixel Values:")
print(pixels)

# Display as grayscale image
plt.figure(figsize=(8, 8))
plt.imshow(pixels, cmap='gray', vmin=0, vmax=255)
```

```
for i in range(8):
    for j in range(8):
        value = int(pixels[i, j])
        plt.text(j, i,str(pixels[i, j]),ha="center",va="center",
            color="black" if pixels[i,j] > 127 else "white"
        )
plt.xticks(range(8))
plt.yticks(range(8))
plt.title("8×8 Image Pixel Values")
plt.grid()
plt.show()
```

Basic Histogram Equalization

```
# Count each intensity value
values, counts = np.unique(pixels,
return_counts=True)

for value, count in zip(values, counts):
    print(f"Pixel Value = {value}, Count = {count}")

# Histogram
    L=256
hist = np.bincount(pixels.ravel(), minlength=L)

print("Histogram:")
print(hist)
```

Basic Histogram Equalization

```
# 8. Four Plots
plt.figure(figsize=(16, 8))

plt.subplot(2,3,1)
plt.imshow(pixels, cmap='gray', vmin=0,
vmax=255)

for i in range(pixels.shape[0]):
    for j in range(pixels.shape[1]):

        value = int(pixels[i, j])

        if value > 127:
            text_color = "black"
        else:
            text_color = "white"
```

```
plt.text(
            j,
            i,
          str(value),
          ha="center",
          va="center",
          color=text_color,
          fontsize=9
        )
```

Basic Histogram Equalization

```
plt.xticks(range(8))
plt.yticks(range(8))
plt.title("Original Image + Pixel Values")

#plt.grid()
plt.subplot(2,3,2)
plt.hist(pixels)

# Probability
pdf = hist / pixels.size

# CDF
cdf = np.cumsum(pdf)
```

```
# Print CDF for values that actually occur
for intensity in np.unique(pixels):
    print(
        f"Intensity = {intensity:3d}, "
        f"Frequency = {hist[intensity]:2d}, "
        f"CDF = {cdf[intensity]:.4f}"
    )
```

Basic Histogram Equalization

```
# Plot CDF
plt.subplot(2,3,3)
plt.plot(cdf)
plt.xlabel("Pixel Intensity")
plt.ylabel("CDF")
plt.title("Cumulative Distribution
Function")
plt.grid()

# Histogram equalization
formula
s = (L - 1) * cdf

# Convert to integer pixel values
s_round =
np.round(s).astype(np.uint8)
```

```
# Apply mapping to image
equalized = s[pixels]

plt.tight_layout()

plt.subplot(2,3,4)
plt.imshow(equalized,
cmap='gray', vmin=0,
vmax=255)
plt.title("Equalized")
for i in
range(equalized.shape[0]):
    for j in
range(equalized.shape[1]):

        value = int(equalized[i, j])
```

Basic Histogram Equalization

```
if value > 127:
        text_color = "black"
    else:
        text_color = "white"

    plt.text(
        j,
        i,
        str(value),
        ha="center",
        va="center",
        color=text_color,
        fontsize=9
    )
```

```
plt.subplot(2,3,5)
plt.hist(equalized)
plt.title("Equalized")

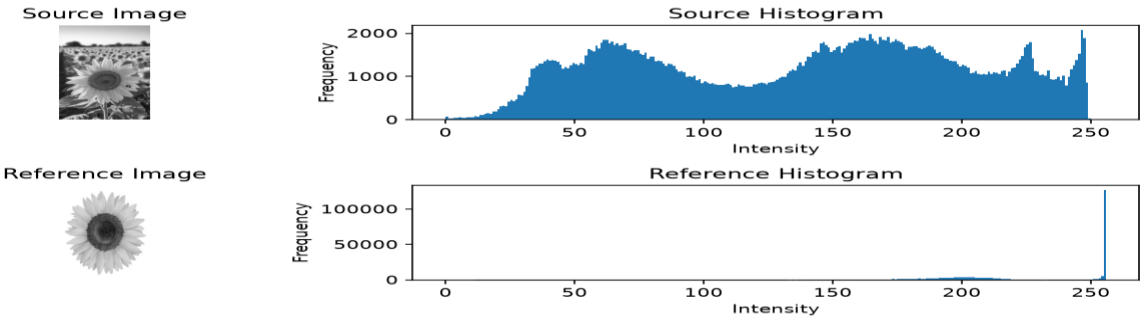
plt.tight_layout()
plt.show()
#eqlized_hist =
np.bincount(equalized.ravel(),
minlength=L)
print("Original:")
print(pixels)

print("\nEqualized:")
print(equalized)
```

Histogram Matching

```
# Image Paths
Img1 = r"D:/DIPLab/Image/sunflower1.jpg"   # Source image
Img2 = r"D:/DIPLab/Image/sunflower2.jpg"   # Reference image

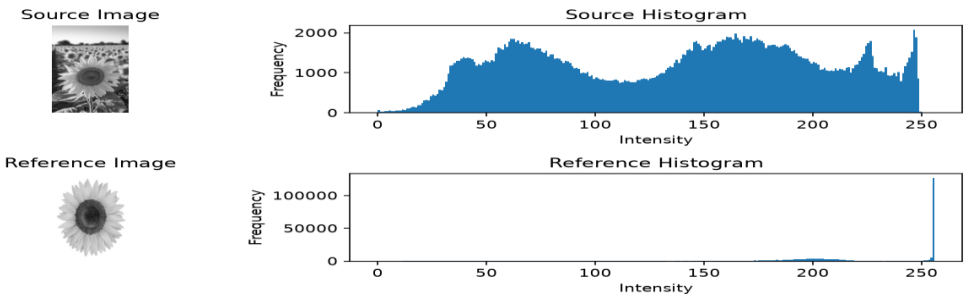
# Check Files
if not os.path.isfile(Img1):
    raise FileNotFoundError(
        f"Source image not found: {Img1}"
    )
if not os.path.isfile(Img2):
    raise FileNotFoundError(
        f"Reference image not found: {Img2}"
    )
```



Histogram Matching

```
# Read Images as Grayscale
sourceImage = cv2.imread(
    Img1,
    cv2.IMREAD_GRAYSCALE
)

referenceImage = cv2.imread(
    Img2,
    cv2.IMREAD_GRAYSCALE
)
```




```
# Check Image Loading
if sourceImage is None:
    raise ValueError("Could not load source image.")
if referenceImage is None:
    raise ValueError("Could not load reference image.")
```

Histogram Matching

```
# Display Original Image Sizes
print("Original Source Shape   :", sourceImage.shape)
print("Original Reference Shape:", referenceImage.shape)

# Resize Reference Image
# Make both images the same size
height, width = sourceImage.shape

referenceImage = cv2.resize(
    referenceImage,
    (width, height)
)

print("After Resize:")
print("Source Shape   :", sourceImage.shape)
print("Reference Shape:", referenceImage.shape)
```

Histogram Matching

Display Images

```
plt.figure(figsize=(10, 4))

plt.subplot(2, 2, 1)
plt.imshow(sourceImage, cmap="gray")
plt.title("Source Image")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(referenceImage, cmap="gray")
plt.title("Reference Image")
plt.axis("off")
```

Histogram Matching

```
# Display Histograms
plt.subplot(2, 2, 2)
plt.hist(sourceImage.ravel(), 256, [0, 256])
plt.title("Source Histogram")
plt.xlabel("Intensity")
plt.ylabel("Frequency")

plt.subplot(2, 2, 4)
plt.hist(referenceImage.ravel(), 256, [0, 256])
plt.title("Reference Histogram")
plt.xlabel("Intensity")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```

Histogram Matching

```
# Pixel Comparison
# Tolerance
tolerance = 5

# Convert to int16 to avoid uint8 overflow
source = sourceImage.astype(np.int16)
reference = referenceImage.astype(np.int16)

# Absolute pixel difference
difference = np.abs(source - reference)

# Find Similar Pixels
# A pixel is similar if difference <= tolerance
similar = difference <= tolerance
```

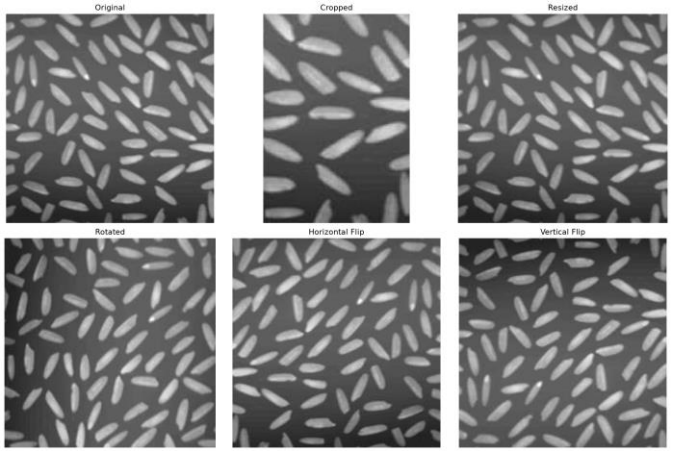
Histogram Matching

```
# Calculate Pixel Statistics
similar_pixels = np.sum(similar)
total_pixels = similar.size
different_pixels = total_pixels - similar_pixels

similarity_percentage = (similar_pixels / total_pixels) * 100

# Display Results
print("       PIXEL SIMILARITY RESULT")
print("Similar pixels  :", similar_pixels)
print("Different pixels:", different_pixels)
print("Total pixels    :", total_pixels)
print("Tolerance       :", tolerance)
print("Similarity (%)  :", similarity_percentage)
```

Module#2.2: Image Transformation: Crop, Resize, Rotate, Flip



Smoothing Filters

1. Average (Mean) Filter
average = cv2.blur(img, (5,5))

2. Gaussian Filter
gaussian = cv2.GaussianBlur(img, (5,5), 0)

3. Median Filter
median = cv2.medianBlur(img, 5)

4. Bilateral Filter
bilateral = cv2.bilateralFilter(img, 9, 75, 75)

Module#2.2: Image Transformation: Crop, Resize, Rotate, Flip

```
# Crop Image
cropped = img[100:350, 150:450]

# Resize Image
resized = cv2.resize(img, (300, 300))

# Rotate Image (90° Clockwise)
rotated = cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)
```

```
# Flip Image
# Horizontal Flip
flip_horizontal = cv2.flip(img, 1)
```

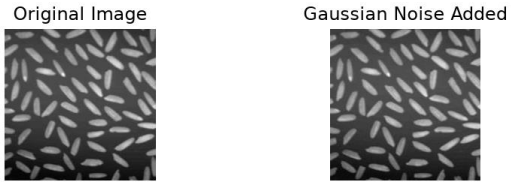
```
# Vertical Flip
flip_vertical = cv2.flip(img, 0)

# Both Horizontal & Vertical
flip_both = cv2.flip(img, -1)
```

Module#3.1: Image Filtering: Gaussian Noise, Mean, Median, Gaussian, Bilateral

```
# Read grayscale image
Img_path=r"D:/DIPLab/Image/rice.tiff";
img = cv2.imread(Img_path,cv2.IMREAD_GRAYSCALE)

# Generate Gaussian noise
mean = 0
std = 2
```




```
noiseImage = np.random.normal(mean,std,img.shape)
print("Image Shape:", img.shape)
print("Pixel Values:")
print(img)
```

```
3×3 Image with Noisy Value:
 [153.96001228  168.88797667  187.23369747]
 [ 90.00280199   93.36473408   93.50453713]
 [128.77036655   64.83717005   63.67267689]
```

Module#3.1: Image Filtering: Gaussian Noise, Mean, Median, Gaussian, Bilateral

```
# Add noise
noisy_img = img.astype(np.float32) + noiseImage
print("Noisy Image Shape:", img.shape)
print("Pixel Values:")
print(img)

# Select a 3×3 region from the top-left corner
ImgPixels_3x3=cv2.resize(img,(3,3))
print("3×3 Pixel Values:")
print(ImgPixels_3x3)

NoiseImageValue=ImgPixels_3x3+noiseValue
print("3×3 Image with Noisy Value:")
print(NoiseImageValue)

# Keep pixel values between 0 and 255
noisy_img = np.clip(noisy_img,0,255).astype(np.uint8)
```

Module#3.1: Mean, Median, Gaussian, Bilateral

Mean (Average) Filter

```
mean = cv2.blur(img, (5,5))
```

Gaussian Filter

```
gaussian = cv2.GaussianBlur(img, (5,5), 0)
```

Median Filter

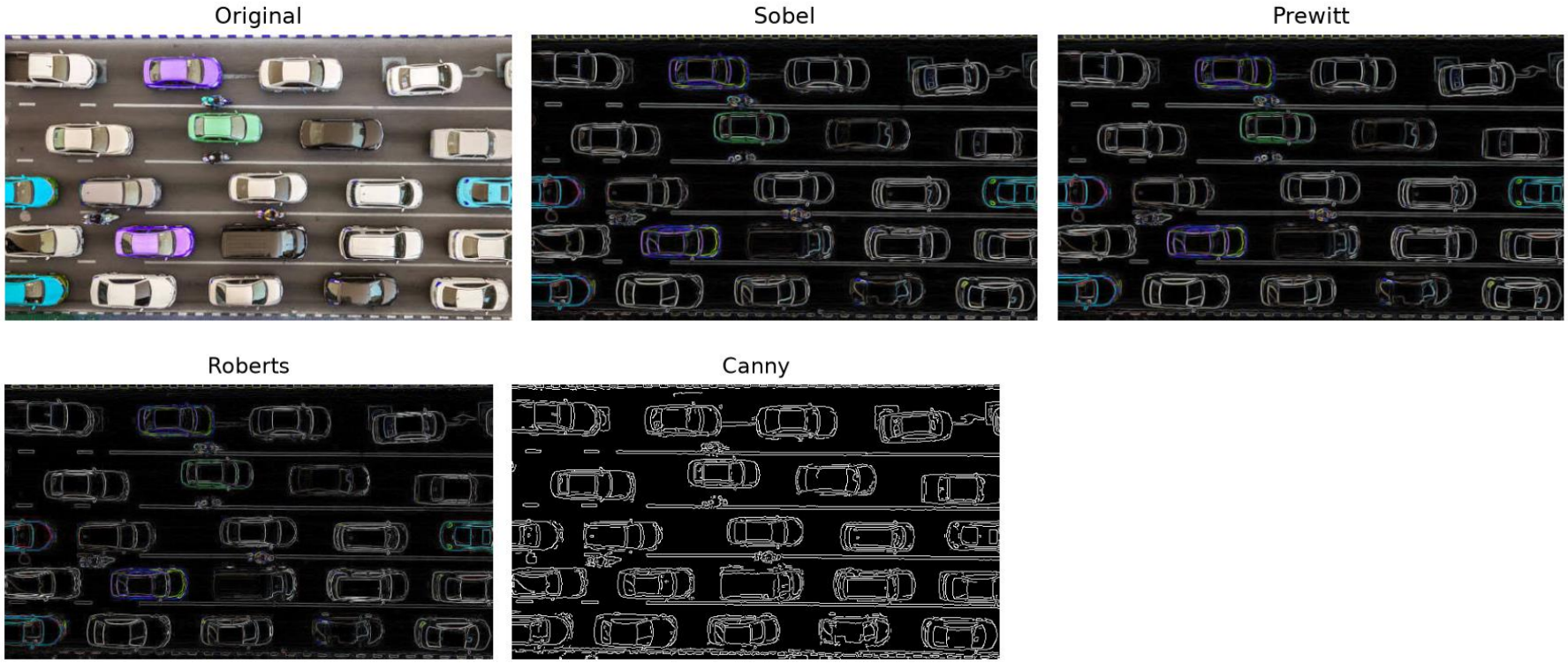
```
median = cv2.medianBlur(img, 5)
```

Bilateral Filter

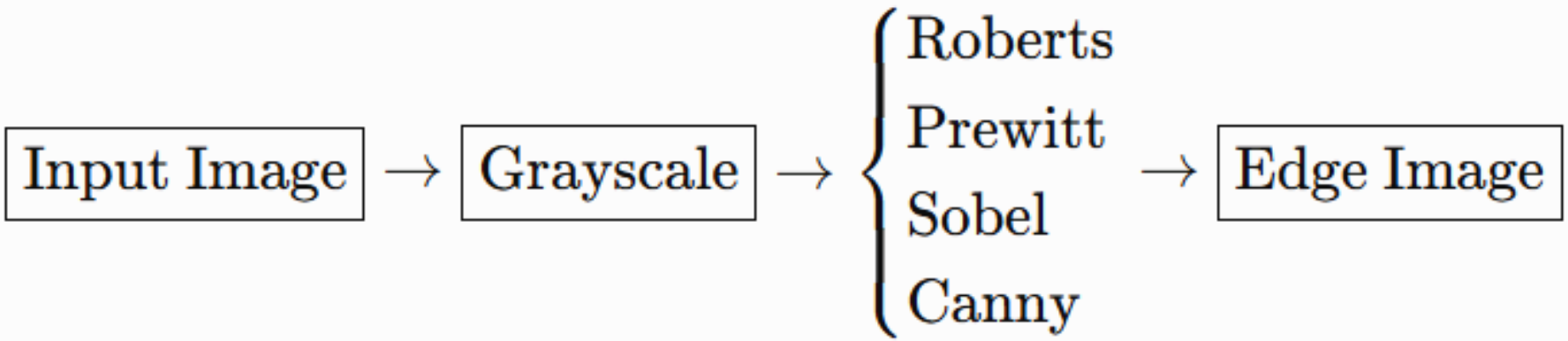
```
bilateral = cv2.bilateralFilter(img, 9, 75, 75)
```

Module#3.2: Edge Detection: Sobel, Prewitt, Roberts, Canny operators, Laplacian

$$\boxed{\text{Input Image}} \rightarrow \boxed{\text{Grayscale}} \rightarrow \begin{cases} \text{Roberts} \\ \text{Prewitt} \\ \text{Sobel} \\ \text{Canny} \end{cases} \rightarrow \boxed{\text{Edge Image}}$$



Module#3.2: Edge Detection: Sobel, Prewitt, Roberts, Canny operators, Laplacian



Sobel

```
sx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sy = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
sobel = cv2.magnitude(sx.astype(np.float32),
                      sy.astype(np.float32))
```

Module#3.2: Edge Detection: Sobel, Prewitt, Roberts, Canny operators, Laplacian

```
# Prewitt
kx = np.array([[-1, 0, 1],
               [-1, 0, 1],
               [-1, 0, 1]], dtype=np.float32)

ky = np.array([[-1, -1, -1],
               [ 0,  0,  0],
               [ 1,  1,  1]], dtype=np.float32)

px = cv2.filter2D(img, cv2.CV_32F, kx)
py = cv2.filter2D(img, cv2.CV_32F, ky)

prewitt = cv2.magnitude(px, py)
```

Module#3.2: Edge Detection: Sobel, Prewitt, Roberts, Canny operators, Laplacian

```
# Roberts
rx_kernel = np.array([[1, 0],
                      [0, -1]], dtype=np.float32)

ry_kernel = np.array([[0, 1],
                      [-1, 0]], dtype=np.float32)

rx = cv2.filter2D(img, cv2.CV_32F, rx_kernel)
ry = cv2.filter2D(img, cv2.CV_32F, ry_kernel)

roberts = cv2.magnitude(rx, ry)
```

Module#3.2: Edge Detection: Sobel, Prewitt, Roberts, Canny operators, Laplacian

```
# Canny
canny = cv2.Canny(img, 100, 200)

# Normalize gradient results for display
sobel = cv2.normalize(sobel, None, 0, 255,
                      cv2.NORM_MINMAX).astype(np.uint8)

prewitt = cv2.normalize(prewitt, None, 0, 255,
                        cv2.NORM_MINMAX).astype(np.uint8)

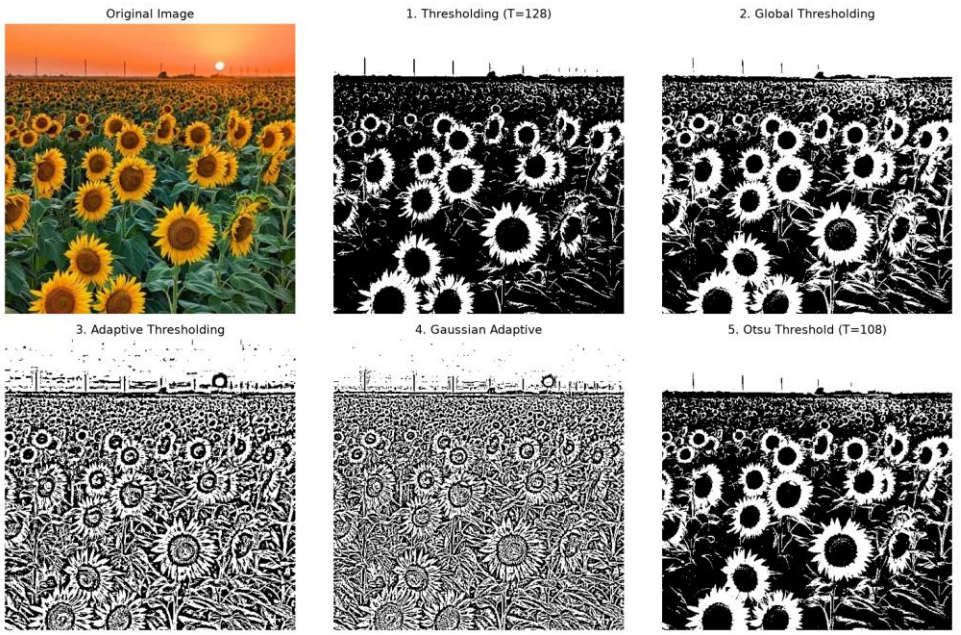
roberts = cv2.normalize(roberts, None, 0, 255,
                        cv2.NORM_MINMAX).astype(np.uint8)
```

Module#3.3: Image Segmentation

1. Thresholding
2. Global Thresholding
3. Adaptive Thresholding
4. Gaussian Adaptive Thresholding
5. Otsu Thresholding

THRESHOLDING RESULTS

Basic Threshold : 128
Global Threshold : 100
Adaptive Threshold : Local
Gaussian Adaptive : Local Gaussian
Otsu Optimal Threshold: 108.0



Module#3.3: Image Segmentation

1. BASIC THRESHOLDING

```
T = 128

_, basic_threshold = cv2.threshold(
    gray,
    T,
    255,
    cv2.THRESH_BINARY)
```

2. GLOBAL THRESHOLDING

```
global_T = 100

_, global_threshold = cv2.threshold(
    gray,
    global_T,
    255,
    cv2.THRESH_BINARY)
```

Module#3.3: Image Segmentation

3. ADAPTIVE THRESHOLDING – MEAN

```
adaptive_threshold = cv2.adaptiveThreshold(
    gray,
    255,
    cv2.ADAPTIVE_THRESH_MEAN_C,
    cv2.THRESH_BINARY,
    11,
    2)
```

4. GAUSSIAN ADAPTIVE THRESHOLDING

```
gaussian_threshold = cv2.adaptiveThreshold(
    gray,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11,
    2)
```

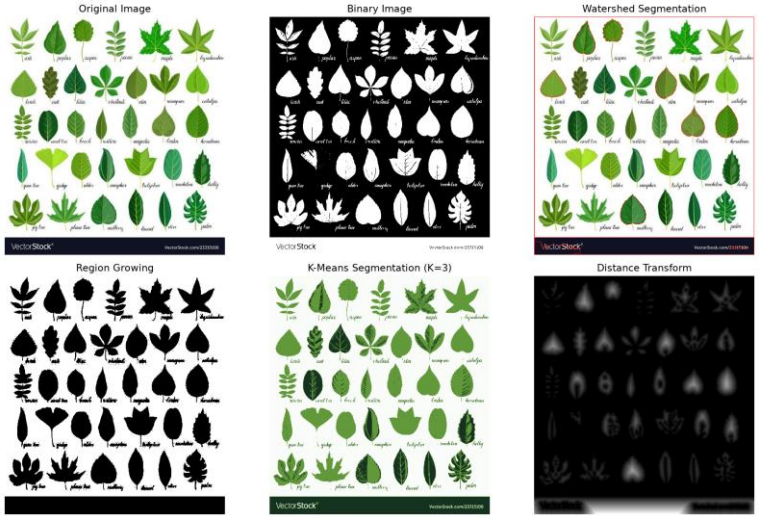
Module#3.3: Image Segmentation

5. OTSU THRESHOLDING

```
otsu_T, otsu_threshold = cv2.threshold(
    gray,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

Module#3.4: Image Segmentation: Watershed, Region Growing, K-Means

```
----------------------------------------
IMAGE SEGMENTATION RESULTS
----------------------------------------
Watershed      : Marker-based segmentation
Region Growing : Seed-based segmentation
K-Means        : Clustering-based
segmentation
K              : 3
Seed Point     : (266, 288)
Region Threshold: 15
----------------------------------------
```




```
# Convert to RGB for displaying
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

Module#3.4: Image Segmentation: Watershed, Region Growing, K-Means

```
# 1. WATERSHED SEGMENTATION
# Otsu thresholding
_, binary = cv2.threshold(
    gray,
    0,
    255,
    cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

                # Morphological opening
                kernel = np.ones((3, 3), np.uint8)
                 opening = cv2.morphologyEx(
                    binary,
                    cv2.MORPH_OPEN,
                    kernel,
                    iterations=2)
```

Module#3.4: Image Segmentation: Watershed, Region Growing, K-Means

```
# Sure background
sure_bg = cv2.dilate(
    opening,
    kernel,
    iterations=3)

# Distance transform
dist_transform =
cv2.distanceTransform(
    opening,
    cv2.DIST_L2, 5)
```

```
# Sure foreground
_, sure_fg = cv2.threshold(
    dist_transform,
    0.5 * dist_transform.max(),
    255,
    0)

sure_fg = np.uint8(sure_fg)

# Unknown region
unknown = cv2.subtract(
    sure_bg,
    sure_fg)
```

Module#3.4: Image Segmentation: Watershed, Region Growing, K-Means

```
# Create markers
num_labels, markers = cv2.connectedComponents(
    sure_fg)

markers = markers + 1

markers[unknown == 255] = 0

# Apply watershed
watershed_img = img.copy()

markers = cv2.watershed(
    watershed_img,
    markers)
```

Module#3.4: Image Segmentation: Watershed, Region Growing, K-Means

```
# Mark boundaries
watershed_img[markers == -1] = [0, 0, 255]

watershed_result = cv2.cvtColor(
    watershed_img,
    cv2.COLOR_BGR2RGB)

# 2. REGION GROWING

# Select seed point
seed_x = gray.shape[1] // 2
seed_y = gray.shape[0] // 2
```

Module#3.4: Image Segmentation: Watershed, Region Growing, K-Means

```
# Similarity threshold
threshold = 15

# Region mask
region = np.zeros_like(gray, dtype=np.uint8)

# Visited pixels
visited = np.zeros_like(gray, dtype=np.uint8)

# Seed intensity
seed_value = int(gray[seed_y, seed_x])

# Stack for region growing
stack = [(seed_x, seed_y)]
 while stack:
    x, y = stack.pop()
```

Module#3.4: Image Segmentation: Watershed, Region Growing, K-Means

```
# Boundary check
    if x < 0 or x >= gray.shape[1]:
        continue

    if y < 0 or y >= gray.shape[0]:
        continue

    # Already visited
    if visited[y, x] == 1:
        continue

    visited[y, x] = 1

    # Similarity condition
    if abs(int(gray[y, x]) - seed_value) <= threshold:

        region[y, x] = 255
```

```
# 4-connected neighbors
        stack.append((x + 1, y))
        stack.append((x - 1, y))
        stack.append((x, y + 1))
        stack.append((x, y - 1))


region_result = region
```

Module#3.4: Image Segmentation: Watershed, Region Growing, K-Means

```
# 3. K-MEANS SEGMENTATION
 # Reshape image pixels
pixels = rgb.reshape((-1, 3))

# Convert to float32
pixels = np.float32(pixels)

# Number of clusters
K = 3

# K-Means termination criteria
criteria = ( cv2.TERM_CRITERIA_EPS +
   cv2.TERM_CRITERIA_MAX_ITER,
   100,    0.2)
```

Module#3.4: Image Segmentation: Watershed, Region Growing, K-Means

```
# Apply K-Means
compactness, labels, centers = cv2.kmeans(
    pixels,
    K,
    None,
    criteria,
    10,
    cv2.KMEANS_RANDOM_CENTERS)

# Convert centers to uint8
centers = np.uint8(centers)

# Replace pixels with cluster centers
segmented_pixels = centers[labels.flatten()]
```

```
# Reshape to original image size
kmeans_result =
segmented_pixels.reshape(
    rgb.shape)
```

Thank You